AI-ASSISTANT-CODING-LAB-21.3

Name: G. Kruthik Roshan
Batch: 41
Roll No: 2303A52339

Collaborative AI Coding: Pair Programming and Version Control Integration

Task 1 – AI-Assisted Pair Programming

Example:

Use AI to generate an initial Expense Tracker Application.

• Ask AI to create a basic program to record daily expenses and

categories.

• Prompt AI to suggest improvements such as:

o Monthly expense summary

o Input validation

o Code refactoring into functions

• Compare the initial and improved versions to understand AI's

role in collaborative coding.

Prompt: "Create a basic Python script to record daily expenses and categories. Then, refactor it into functions, add input validation to prevent negative amounts, and include a function that prints a monthly summary."

Gemini

```python
expenses = []
categories = set()

def record_expense():
    try:
        amount = float(input("Enter expense amount: "))
        description = input("Enter expense description: ")
        category = input("Enter expense category: ").lower()

        expenses.append({'amount': amount, 'description': description, 'category': category})
        categories.add(category)
        print("Expense recorded successfully!")
    except ValueError:
        print("Invalid amount. Please enter a number.")

def view_expenses():
    if not expenses:
        print("No expenses recorded yet.")
        return

    print("\n--- All Expenses ---")
```

```python
        print("No expenses recorded yet.")
        return

    print("\n--- All Expenses ---")
    for i, expense in enumerate(expenses):
        print(f"{i+1}. Amount: ${expense['amount']:.2f}, Description: {expense['description']}, Cat
    print("--------------------\n")

def main():
    while True:
        print("\nExpense Tracker Menu:")
        print("1. Record a new expense")
        print("2. View all expenses")
        print("3. Exit")

        choice = input("Enter your choice: ")

        if choice == '1':
            record_expense()
        elif choice == '2':
            view_expenses()
        elif choice == '3':
```

Terminal

```python
            print("Exiting Expense Tracker. Goodbye!")
            break
        else:
            print("Invalid choice. Please try again.")

if __name__ == "__main__":
    main()
```

```
Expense Tracker Menu:
1. Record a new expense
2. View all expenses
3. Exit
Enter your choice: Record a new expense
Invalid choice. Please try again.

Expense Tracker Menu:
1. Record a new expense
2. View all expenses
3. Exit
Enter your choice: 1
Enter expense amount: 78900
Enter expense description: Sevnty Eight Thousand Nine Hundred
Enter expense category: Salary
```

```
Expense Tracker Menu:
1. Record a new expense
2. View all expenses
3. Exit
Enter your choice: Record a new expense
Invalid choice. Please try again.

Expense Tracker Menu:
1. Record a new expense
2. View all expenses
3. Exit
Enter your choice: 1
Enter expense amount: 78900
Enter expense description: Sevnty Eight Thousand Nine Hundred
Enter expense category: Salary
Expense recorded successfully!

Expense Tracker Menu:
1. Record a new expense
2. View all expenses
3. Exit
Enter your choice: Exit
```

```
Expense Tracker Menu:
1. Record a new expense
2. View all expenses
3. Exit
Enter your choice: 2

--- All Expenses ---
1. Amount: $78900.00, Description: Sevnty Eight Thousand Nine Hundred, Category: salary
--------------------
```

Justification: I started with a core script to establish the logic, then used AI to modularize the code. This shows how AI acts as a 'senior dev' by cleaning up structure and ensuring the app doesn't crash on bad user input.

Task 2 – Version Control Integration

Example:

Manage a Student Grade Calculator project using Git.

• Initialize a Git repository for the project.

• Create a new branch to add AI-assisted features such as:

o Grade classification (A, B, C, etc.)

o Average and percentage calculation

• Commit changes with meaningful commit messages.

• Merge the feature branch back into the main branch and resolve

conflicts if any.

Prompt: "I have a basic grade calculator; help me write a function that calculates the average and assigns a letter grade (A-F). Also, provide the Git commands to create a branch named feature-grading and merge it back."

Gemini

```
students = {}

def add_student_grade():
    name = input("Enter student name: ")
    try:
        grade = float(input(f"Enter {name}'s grade: "))
        if 0 <= grade <= 100:
            students[name] = grade
            print(f"Grade for {name} added successfully!")
        else:
            print("Grade must be between 0 and 100.")
    except ValueError:
        print("Invalid grade. Please enter a number.")

def view_grades():
    if not students:
        print("No student grades recorded yet.")
        return
```

```python
def main_grade_calculator():
    while True:
        print("\nStudent Grade Calculator Menu:")
        print("1. Add student grade")
        print("2. View all grades")
        print("3. Exit")

        choice = input("Enter your choice: ")

        if choice == '1':
            add_student_grade()
        elif choice == '2':
            view_grades()
        elif choice == '3':
            print("Exiting Grade Calculator. Goodbye!")
            break
        else:
            print("Invalid choice. Please try again.")

if __name__ == "__main__":
    main_grade_calculator()
```

```
Student Grade Calculator Menu:
1. Add student grade
2. View all grades
3. Exit
Enter your choice: 1
Enter student name: Alice
Enter Alice's grade: 98
Grade for Alice added successfully!

Student Grade Calculator Menu:
1. Add student grade
2. View all grades
3. Exit
Enter your choice: 2

--- Student Grades ---
Student: Alice, Grade: 98.00
----------------------
```

Justification: This approach uses AI to bridge the gap between "writing code" and "managing a project." It helps keep the main branch stable while we experiment with new logic in an isolated environment.

Task 3 – AI for Commit Message Generation

Example:

Use AI to generate commit messages for a Password Validation

Module.

• Modify the code to add rules like:

o Minimum length

o Special character requirement

• Ask AI to generate commit messages describing these updates.

• Compare AI-generated messages with human-written ones and

analyze clarity.

Prompt: "I just updated my password validator to require at least 8 characters and one special symbol. Based on these changes, write a professional, concise Git commit message following Conventional Commits standards."

```python
def validate_password(password):
    """Validates a password based on basic criteria."""
    if len(password) < 8:
        return False, "Password must be at least 8 characters long."
    if not any(char.isdigit() for char in password):
        return False, "Password must contain at least one digit."
    if not any(char.isupper() for char in password):
        return False, "Password must contain at least one uppercase letter."
    if not any(char.islower() for char in password):
        return False, "Password must contain at least one lowercase letter."
    return True, "Password is valid."

# Example usage:
if __name__ == "__main__":
    print("--- Password Validation Test ---")

    # Test cases
    password_strength = [
        ("Short1@", False, "Password must be at least 8 characters long."), # Changed password to b
        ("Nodigit@A", False, "Password must contain at least one digit."),
        ("noupper1@a", False, "Password must contain at least one uppercase letter."),
```

Terminal

```python
# Test cases
password_strength = [
    ("Short1@", False, "Password must be at least 8 characters long."), # Changed password to b
    ("Nodigit@A", False, "Password must contain at least one digit."),
    ("noupper1@a", False, "Password must contain at least one uppercase letter."),
    ("NOLOWER1@A", False, "Password must contain at least one lowercase letter."),
    ("ValidP@ss1", True, "Password is valid."),
    ("another_valid_password_2", False, "Password must contain at least one uppercase letter.")
]

for pwd, expected_valid, expected_msg in password_strength:
    is_valid, message = validate_password(pwd)
    print(f"Password: '{pwd}'\n  Valid: {is_valid} (Expected: {expected_valid})\n  Message: {me

    if is_valid != expected_valid or message != expected_msg:
        print("  TEST FAILED for password: ", pwd)
    else:
        print("  TEST PASSED")
    print("--------------------------------")
```

```
--- Password Validation Test ---
Password: 'Short1@'
  Valid: False (Expected: False)
  Message: Password must be at least 8 characters long. (Expected: Password must be at least 8 charac

  TEST PASSED
--------------------------------
Password: 'Nodigit@A'
  Valid: False (Expected: False)
  Message: Password must contain at least one digit. (Expected: Password must contain at least one di

  TEST PASSED
--------------------------------
Password: 'noupper1@a'
  Valid: False (Expected: False)
  Message: Password must contain at least one uppercase letter. (Expected: Password must contain at l

  TEST PASSED
--------------------------------
Password: 'NOLOWER1@A'
  Valid: False (Expected: False)
  Message: Password must contain at least one lowercase letter. (Expected: Password must contain at l

  TEST PASSED
```

Justification: Human messages are often vague (like "fixed stuff"), but AI can instantly generate descriptive, standardized messages. This improves project documentation and makes it much easier for teammates to **track history.**

Task 4 – Pair Programming Simulation

Scenario:

Collaboratively develop a Quiz Management System.

• Student A creates the initial quiz program with questions and

answers.

• Student B, with AI assistance, adds features such as:

o Score calculation

o Input validation

• Both students collaborate using Git branching and merging to

integrate their changes.

Prompt: "Act as my pair programmer. I have a basic quiz list; please write a robust score-tracking system and a loop that validates if the user entered a valid option (A, B, or C) before proceeding."

```python
class Quiz:
    def __init__(self, questions):
        self.questions = questions
        self.score = 0

    def run_quiz(self):
        print("\n--- Welcome to the Quiz! ---")
        for i, q in enumerate(self.questions):
            print(f"\nQuestion {i+1}: {q['question']}")
            for j, option in enumerate(q['options']):
                print(f"{j+1}. {option}")

            user_answer = input("Your answer (enter the number): ")
            # Basic validation (Student B will improve this)
            if user_answer.isdigit() and 1 <= int(user_answer) <= len(q['options']):
                if q['options'][int(user_answer)-1] == q['answer']:
                    print("Correct!")
                    self.score += 1 # Student B will improve score calculation
                else:
```

```python
            if user_answer.isdigit() and 1 <= int(user_answer) <= len(q['opti
                if q['options'][int(user_answer)-1] == q['answer']:
                    print("Correct!")
                    self.score += 1 # Student B will improve score calculation
                else:
                    print(f"Wrong! The correct answer was: {q['answer']}")
            else:
                print("Invalid input. Skipping this question.")

        print("\n--- Quiz End ---")
        print(f"You scored {self.score} out of {len(self.questions)} questions.")

# Define some quiz questions
quiz_questions = [
    {
        'question': "What is the capital of France?",
        'options': ["Berlin", "Madrid", "Paris", "Rome"],
        'answer': "Paris"
    },
    {
        'question': "Which planet is known as the Red Planet?",
        'options': ["Earth", "Mars", "Jupiter", "Venus"],
```

```python
    {
        'question': "What is the capital of France?",
        'options': ["Berlin", "Madrid", "Paris", "Rome"],
        'answer': "Paris"
    },
    {
        'question': "Which planet is known as the Red Planet?",
        'options': ["Earth", "Mars", "Jupiter", "Venus"],
        'answer': "Mars"
    },
    {
        'question': "What is 2 + 2?",
        'options': ["3", "4", "5", "6"],
        'answer': "4"
    }
]

# Create a quiz instance and run it
if __name__ == "__main__":
    my_quiz = Quiz(quiz_questions)
    my_quiz.run_quiz()
```

```
--- Welcome to the Quiz! ---

Question 1: What is the capital of France?
1. Berlin
2. Madrid
3. Paris
4. Rome
Your answer (enter the number): 1
Wrong! The correct answer was: Paris

Question 2: Which planet is known as the Red Planet?
1. Earth
2. Mars
3. Jupiter
4. Venus
Your answer (enter the number): 4
Wrong! The correct answer was: Mars

Question 3: What is 2 + 2?
1. 3
2. 4
3. 5
4. 6
```

```
Question 3: What is 2 + 2?
1. 3
2. 4
3. 5
4. 6
Your answer (enter the number): 2
Correct!

--- Quiz End ---
You scored 1 out of 3 questions.
```

Justification:

This simulation highlights the 'driver-navigator' dynamic. While one student handles the core logic, the AI (assisting Student B) focuses on "defensive programming," ensuring the system is user-friendly and error-free.